Facultad de Ingeniería

Escuela de Ingeniería en Computación

Asignatura: Desarrollo de Aplicaciones con Web Frameworks DWF404 G01T

Ciclo / Grupo: 03– 2026 / 01T

Gestión de Archivos y Almacenamiento en la Nube

Facilitador: Ing. Bladimir Diaz Campos

Elaborado por:

Paola Matilde Orellana Castillo	OC250609
Aleshka Priscila Gámez Navidad	GN252409
Gabriela Giselle Vázquez Rodrígez	VR250295
Mariana Maytee López Gómez	LG252169

Gestión de Archivos y Almacenamiento en la Nube 3
Introducción 3
Desarrollo 3
Uso de Docker y LocalStack 3
Configuración de la conexión con S3 4
Creación del bucket 7
Verificación de la conexión ..... **¡Error! Marcador no definido.**
Archivo de configuración 9
Endpoint para subir archivos 10
Procesamiento del archivo (S3Service) 11
Desarrollo : Implementación del endpoint de descarga 13
1. Búsqueda del archivo en el Bucket (Capa de Servicio) 13
2. Retorno al usuario y manejo de errores (Capa de Controlador) 13
3. Manejo de Errores (Archivos no encontrados) 14
Desarrollo del Endpoint de Eliminación 15
Pruebas realizadas y validación 16

Gestión de Archivos y Almacenamiento en la Nube

Introducción

Muchas aplicaciones necesitan almacenar archivos como imágenes, documentos o videos de forma segura y accesible desde cualquier lugar. Estos archivos se guardaban en el servidor local, lo que puede generar limitaciones en almacenamiento, seguridad y escalabilidad.

Por esta razón, surge la necesidad de utilizar servicios en la nube, los cuales permiten gestionar archivos de manera más eficiente. En este proyecto se implementa una solución que conecta una aplicación desarrollada en Spring Boot con el servicio de almacenamiento S3 de Amazon Web Services.

Además, para facilitar el desarrollo y evitar costos, se utilizó Docker junto con LocalStack, una herramienta que simula los servicios de AWS en un entorno local. Esto permitió realizar pruebas reales de funcionamiento sin necesidad de conectarse directamente a la nube.

Con esta implementación, el sistema queda preparado para manejar archivos en la nube, permitiendo su futura subida, descarga y eliminación de forma eficiente.

Desarrollo

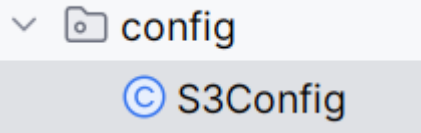
Uso de Docker y LocalStack

Para simular el servicio de almacenamiento en la nube, se utilizó Docker ejecutando LocalStack. Esta herramienta permite emular los servicios de Amazon Web Services en un entorno local, facilitando las pruebas sin necesidad de utilizar recursos reales en la nube.

Docker se encarga de ejecutar LocalStack como un contenedor independiente del proyecto, el cual expone un puerto local (4566) donde la aplicación puede conectarse. De esta manera, la aplicación Spring Boot interactúa con un servicio simulado que se comporta de forma similar a S3.

Configuración de la conexión con S3

Para establecer la conexión con S3, se creó una clase de configuración en Spring Boot:




```
@Configuration
public class S3Config {
```

La clase S3Config se encarga de definir la configuración necesaria para establecer la conexión con el servicio de almacenamiento en la nube. Se utiliza la anotación @Configuration, lo que indica que esta clase contiene configuraciones que serán gestionadas por el framework Spring.

Dentro de esta clase se definen varias variables que almacenan los datos necesarios para la conexión, como la región, el endpoint y las credenciales de acceso. Estos valores no están escritos directamente en el código, sino que se obtienen desde el archivo de configuración mediante la anotación @Value.

La región indica la ubicación del servicio de almacenamiento, mientras que el endpoint define la dirección a la que se conectará la aplicación. En este caso, el endpoint apunta a un entorno local donde se está ejecutando el servicio simulado. Por su parte, las credenciales permiten autenticar la conexión, aunque en este entorno se utilizan valores de prueba.

Este enfoque permite mantener el código más flexible y fácil de mantener, ya que los valores pueden modificarse según el entorno sin necesidad de realizar cambios en la lógica de la aplicación.

```java
@Value("${aws.region}")
    private String region;

    @Value("${aws.endpoint}")
    private String endpoint;

    @Value("${aws.accessKey}")
    private String accessKey;

    @Value("${aws.secretKey}")
    private String secretKey;
```

Después, se creo el cliente de S3:

```java
@Bean
    public S3Client s3Client() {
        return S3Client.builder()
                .region(Region.of(region))
                .endpointOverride(URI.create(endpoint))
                .serviceConfiguration(
                        S3Configuration.builder()
                                .pathStyleAccessEnabled(true)
                                .build()
                )
                .credentialsProvider(
                        StaticCredentialsProvider.create(
                                AwsBasicCredentials.create(accessKey, secretKey)
                        )
                )
                .build();
    }
}
```

Es el encargado de comunicarse con S3.

- La región define la ubicación del servicio
- El endpoint permite conectarse a LocalStack en lugar de AWS real

- Las credenciales se usan para autenticación (en este caso son de prueba)

La clase S3Config define la configuración necesaria para establecer la conexión con el servicio de almacenamiento. Se utiliza la anotación @Configuration para indicar que contiene definiciones de beans que serán gestionados por Spring.

Los valores como región, endpoint y credenciales se obtienen desde el archivo application.properties, lo que permite una configuración flexible y reutilizable. Esto es especialmente útil para cambiar entre un entorno local y uno en la nube sin modificar el código fuente.

El cliente de S3 se construye mediante un patrón builder, donde se especifica la región del servicio y el endpoint. Este último es clave, ya que permite redirigir la conexión hacia LocalStack en lugar de AWS real.

Adicionalmente, se habilita una configuración especial requerida por LocalStack para el manejo de rutas, y se definen credenciales básicas necesarias para autenticar la conexión. Finalmente, se construye el cliente, quedando disponible para ser utilizado en otras partes de la aplicación.

Luego modificamos S3TestContrroller:

```
@PostMapping("/upload")
    public ResponseEntity<String> uploadFile(@RequestParam("file") MultipartFile file) {
        try {
            String fileName = s3Service.uploadFile(file);
            return ResponseEntity.ok("Archivo subido: " + fileName);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("Error al subir archivo: " + e.getMessage());
        }
    }
}
```

Este método corresponde al endpoint **/upload** y es el encargado de recibir archivos desde el cliente y coordinar su almacenamiento a través del servicio.

Utiliza la anotación:

```
@PostMapping("/upload")
```

lo que indica que solo acepta peticiones **POST**, y recibe el archivo enviado mediante:

```
@RequestParam("file") MultipartFile file
```

Aquí el archivo es capturado desde la solicitud HTTP y enviado al backend.

Dentro del método, se llama al servicio que realiza la lógica de guardado:

```
String fileName = s3Service.uploadFile(file);
```

Este servicio procesa el archivo y devuelve el nombre con el que fue almacenado.

Si todo sale correctamente, el servidor responde con un mensaje de éxito:

```
return ResponseEntity.ok("Archivo subido: " + fileName);
```

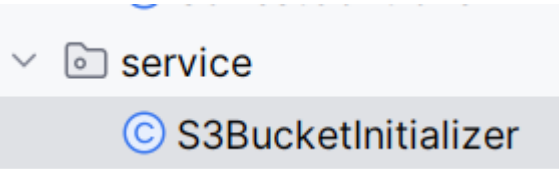
En caso de error durante el proceso, se captura la excepción y se devuelve un mensaje indicando el problema:

```
return ResponseEntity.badRequest().body("Error al subir archivo: " + e.getMessage());
```

Este endpoint permite la **recepció de archivos desde el cliente**, los envía al servicio de almacenamiento y devuelve una respuesta indicando si la operación fue exitosa o no.

Creación del bucket

Para almacenar los archivos, es necesario crear un bucket en S3. Creamos el paquete service y la clase S3BucketInitializer



Esto se realiza automáticamente al iniciar la aplicación:

```java
package sv.edu.udb.gestion_archivos.service;

import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;

@Service
public class S3BucketInitializer {

    private final S3Client s3Client;

    @Value("${aws.bucket.name}")
    private String bucketName;

    @Value("${aws.init.bucket:true}")
    private boolean initBucket;

    public S3BucketInitializer(S3Client s3Client) {
        this.s3Client = s3Client;
    }
```

```java
    @PostConstruct
    public void init() {

        if (!initBucket) return;

        try {
            s3Client.createBucket(CreateBucketRequest.builder()
                    .bucket(bucketName)
                    .build());

            System.out.println("Bucket creado: " + bucketName);

        } catch (Exception e) {
            System.out.println("El bucket ya existe o error: " + e.getMessage());
        }
    }
}
```

La clase `S3BucketInitializer` tiene como propósito garantizar que el espacio de almacenamiento en la nube esté disponible antes de que la aplicación comience a operar. Para ello, se define como un servicio de Spring mediante la anotación `@Service`, lo que permite que sea gestionada automáticamente por el framework.

Dentro de esta clase se utiliza un cliente de S3, el cual ya ha sido configurado previamente y es inyectado a través del constructor. Esto permite reutilizar la conexión establecida con el servicio de almacenamiento sin necesidad de crear nuevas instancias.

El nombre del bucket se obtiene desde el archivo de configuración utilizando la anotación `@Value`, lo que facilita su modificación sin necesidad de alterar el código. Asimismo, se incluye una propiedad adicional que permite controlar si el bucket debe crearse automáticamente al iniciar la aplicación.

El método principal de esta clase está marcado con `@PostConstruct`, lo que indica que se ejecutará automáticamente una vez que la aplicación haya finalizado su proceso de inicialización. En este punto, se verifica si la creación del bucket está habilitada; en caso contrario, no se realiza ninguna acción.

Si la creación está activada, se intenta crear el bucket en el servicio S3 utilizando el nombre previamente definido. En caso de que el bucket ya exista o se produzca algún error, la excepción es controlada para evitar que la aplicación falle, mostrando un mensaje informativo en consola.

De esta manera, se asegura que el entorno de almacenamiento esté correctamente preparado, permitiendo que las demás funcionalidades relacionadas con la gestión de archivos puedan ejecutarse sin inconvenientes.

Archivo de configuración

La aplicación utiliza el archivo application.properties para definir los parámetros necesarios para la conexión con el servicio de almacenamiento en la nube

```
spring.application.name=gestion-archivos
server.port=8080

aws.accessKey=test
aws.secretKey=test
aws.region=us-east-1
aws.endpoint=http://localhost:4566
aws.bucket.name=archivos-grupo6
```

El archivo application.properties centraliza todos los parámetros necesarios para la conexión con el servicio de almacenamiento.

En este caso, se definen valores como el nombre de la aplicación, el puerto de ejecución, las credenciales de acceso, la región, el endpoint y el nombre del bucket. Esto permite mantener el código limpio y facilita la modificación de la configuración según el entorno en el que se ejecute la aplicación.

El uso de un endpoint local indica que la conexión se realiza contra un servicio simulado, mientras que las credenciales utilizadas son de prueba, lo cual es adecuado para entornos de desarrollo.

Endpoint para subir archivos

El sistema implementa un endpoint REST que permite la **carga de archivos desde el cliente hacia el servidor**, el cual posteriormente los almacena en un servicio de almacenamiento en la nube (Amazon S3 o un entorno simulado como LocalStack).

El endpoint definido es:

```
POST /upload
```

Este endpoint forma parte de la API del backend desarrollada en **Spring Boot** y está diseñado para recibir archivos mediante solicitudes HTTP.

El endpoint utiliza el método HTTP:

- **POST** → porque se envían datos (archivos) al servidor
- Tipo de contenido:

```
multipart/form-data
```

Este tipo de petición permite enviar archivos binarios junto con datos adicionales en una sola solicitud.

En Spring Boot, los archivos enviados desde el cliente se reciben utilizando la clase:

Es una interfaz que representa un archivo recibido en una solicitud HTTP. Permite manipular el archivo sin necesidad de guardarlo previamente en el sistema.

El endpoint principal /upload recibe el archivo desde el cliente y lo procesa dentro del contenedor sin afectar la lógica interna de la aplicación.

En este punto el sistema realiza las siguientes acciones:

- Valida si el archivo recibido no está vacío.
- Obtiene el nombre original del archivo enviado por el cliente.
- Genera un nombre único para evitar conflictos dentro del bucket.
- Prepara el archivo para ser enviado al servicio de almacenamiento en la nube.

Procesamiento del archivo (S3Service)

Creamos la clase llamada S3Service:

```java
package sv.edu.udb.gestion_archivos.service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;

@Service
public class S3Service {

    private final S3Client s3Client;

    @Value("${aws.bucket.name}")
    private String bucketName;

    public S3Service(S3Client s3Client) {
        this.s3Client = s3Client;
    }

    public String uploadFile(MultipartFile file) throws Exception {

        String fileName = System.currentTimeMillis() + "_" + file.getOriginalFilename();
        PutObjectRequest request = PutObjectRequest.builder()
                .bucket(bucketName)
                .key(fileName)
                .build();

        s3Client.putObject(
                request,
                software.amazon.awssdk.core.sync.RequestBody.fromBytes(file.getBytes())
        );
        return fileName;
    }
}
```

Es el componente encargado de **procesar y almacenar archivos enviados desde el backend**. Su función principal es recibir un archivo mediante MultipartFile, prepararlo y enviarlo al sistema de almacenamiento configurado.

Primero, el servicio genera un nombre único para el archivo usando la hora actual para evitar duplicados:

```
String fileName = System.currentTimeMillis() + "_" + file.getOriginalFilename();
```

Luego, se crea la solicitud de almacenamiento indicando el bucket donde se guardará el archivo y el nombre con el que será identificado:

```
PutObjectRequest request = PutObjectRequest.builder()
        .bucket(bucketName)
        .key(fileName)
        .build();
```

Después, el archivo se convierte a formato binario y se envía al almacenamiento mediante el cliente S3Client:

```
s3Client.putObject(
        request,
        RequestBody.fromBytes(file.getBytes())
);
```

Para finalizar, el método retorna el nombre del archivo guardado.

Desarrollo : Implementación del endpoint de descarga

Para la funcionalidad de recuperar archivos desde la nube, se creó el endpoint GET /files/{name}. El desarrollo se dividió en dos partes principales: la lógica de búsqueda en el servicio y la exposición de la ruta web en el controlador.

1. Búsqueda del archivo en el Bucket (Capa de Servicio)

Aquí nos conectamos directamente con Amazon S3 para extraer el archivo solicitado.

Código implementado:

```java
public byte[] downloadFile(String fileName) {
   GetObjectRequest request = GetObjectRequest.builder()
           .bucket(bucketName)
           .key(fileName)
           .build();

   return s3Client.getObject(request, ResponseTransformer.toBytes()).asByteArray();
}
```

Explicación línea por línea:

- **GetObjectRequest.builder():** Crea el objeto con las instrucciones de búsqueda para AWS.
- **.bucket(bucketName) y .key(fileName):** Le indica a la API exactamente en qué contenedor buscar y el nombre específico del archivo que el usuario quiere descargar.
- **ResponseTransformer.toBytes():** Se encarga de descargar el archivo y transformarlo directamente a un arreglo de bytes (byte[]). Esto es vital para transferir archivos binarios (como imágenes o PDFs) de forma rápida y segura hacia la siguiente capa.

2. Retorno al usuario y manejo de errores (Capa de Controlador)

Aquí exponemos la ruta web y configuramos el comportamiento del navegador del usuario.

Código implementado:

```java
@GetMapping("/{fileName}")
public ResponseEntity<?> downloadFile(@PathVariable String fileName) {
    try {
        byte[] fileBytes = s3Service.downloadFile(fileName);

        return ResponseEntity.ok()
                .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"" + fileName + "\"")
                .contentType(MediaType.APPLICATION_OCTET_STREAM)
                .body(fileBytes);

    } catch (NoSuchKeyException e) {
        return ResponseEntity.status(404).body("Error: El archivo no existe en la nube.");
    } catch (Exception e) {
        return ResponseEntity.status(500).body("Error interno: " + e.getMessage());
    }
}
```

Activar Wind
Ve a Configuraci

Explicación línea por línea:

- **@GetMapping("/{fileName}"):** Define la ruta de acceso. Usamos @PathVariable para extraer automáticamente el nombre del archivo desde la URL.
- **HttpHeaders.CONTENT_DISPOSITION con "attachment":** Esta es la instrucción clave. Le da una orden directa al navegador del usuario para que fuerce la descarga del archivo en su computadora, en lugar de intentar mostrarlo en la pantalla.
- **MediaType.APPLICATION_OCTET_STREAM:** Define el tipo de contenido como un archivo binario genérico.

3. Manejo de Errores (Archivos no encontrados)

Para evitar que el sistema colapse si el usuario pide un archivo que no está en el bucket, se implementó un control de excepciones:

- **catch (NoSuchKeyException e):** Atrapa el error específico que lanza Amazon S3 cuando el archivo no existe.
- **ResponseEntity.status(404):** Retorna un código HTTP 404 limpio y un mensaje claro ("El archivo no existe en la nube"), protegiendo al usuario de ver trazas de error técnicas (stacktraces).

Desarrollo del Endpoint de Eliminación

Para cumplir con los requisitos de gestión de almacenamiento, se implementó un endpoint encargado de remover objetos de forma permanente del bucket de S3 configurado en LocalStack.

```
Endpoint: DELETE
/api/archivos/eliminar/{nombre}.
```

Proceso en S3: El sistema recibe el nombre del archivo, construye una solicitud de tipo DeleteObjectRequest especificando el bucket y la llave (key), y la envía al cliente de Amazon SDK para su procesamiento.

Confirmación al usuario: Si la operación es exitosa, el sistema devuelve un mensaje de confirmación; de lo contrario, captura la excepción y notifica al usuario que el archivo no pudo ser localizado o eliminado.

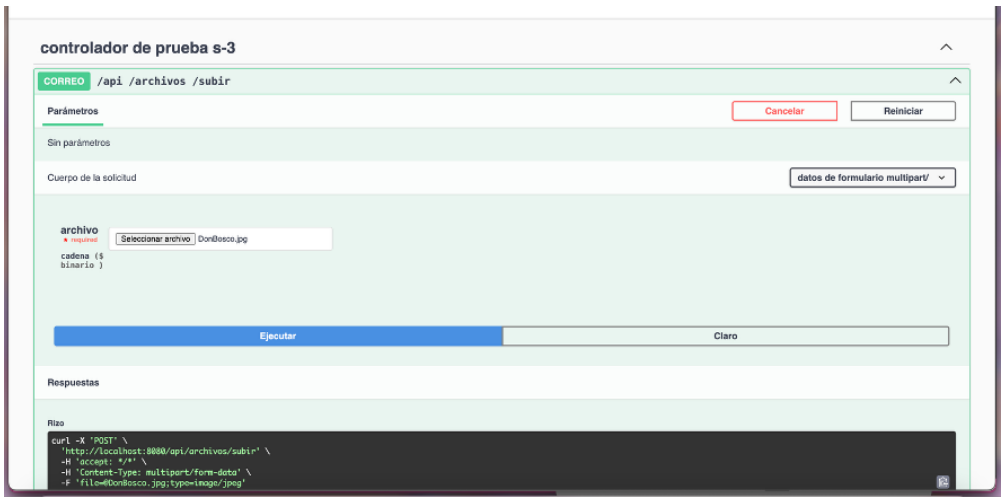
```
    @DeleteMapping("/eliminar/{nombre}")
    public ResponseEntity<String> eliminar(@PathVariable String nombre)
{       String resultado = s3Service.deleteFile(nombre);
        if (resultado.contains("Error")) {
            return ResponseEntity.status(404).body(resultado);
        }
        return ResponseEntity.ok(resultado);
    }
}
```

Pruebas realizadas y validación

Se realizaron pruebas de extremo a extremo (End-to-End) para validar que el flujo de gestión de archivos funciona correctamente sobre la arquitectura de microservicios y contenedores.

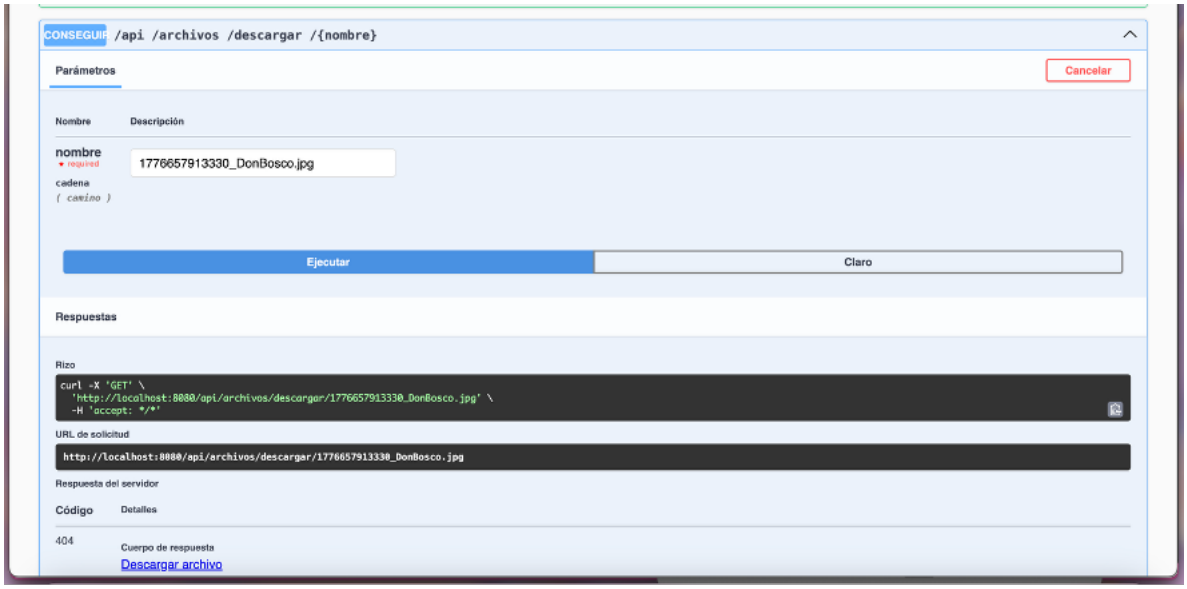
Prueba de Subida (Upload)

Se validó la carga de archivos mediante el protocolo multipart/form-data, confirmando que el archivo se almacena con un identificador único en el servidor local.



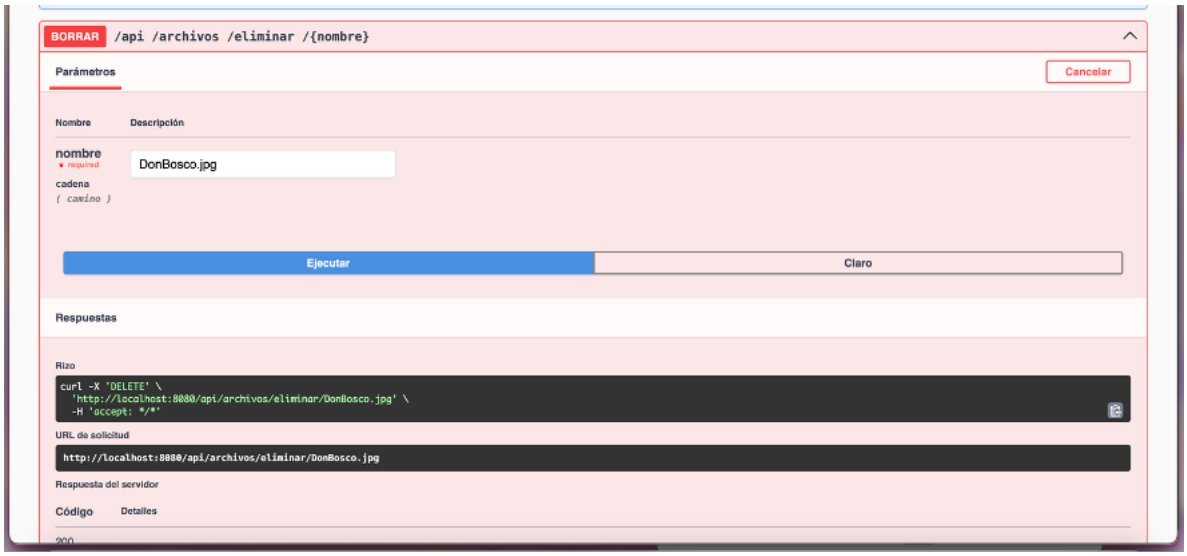
Prueba de Descarga (Download)

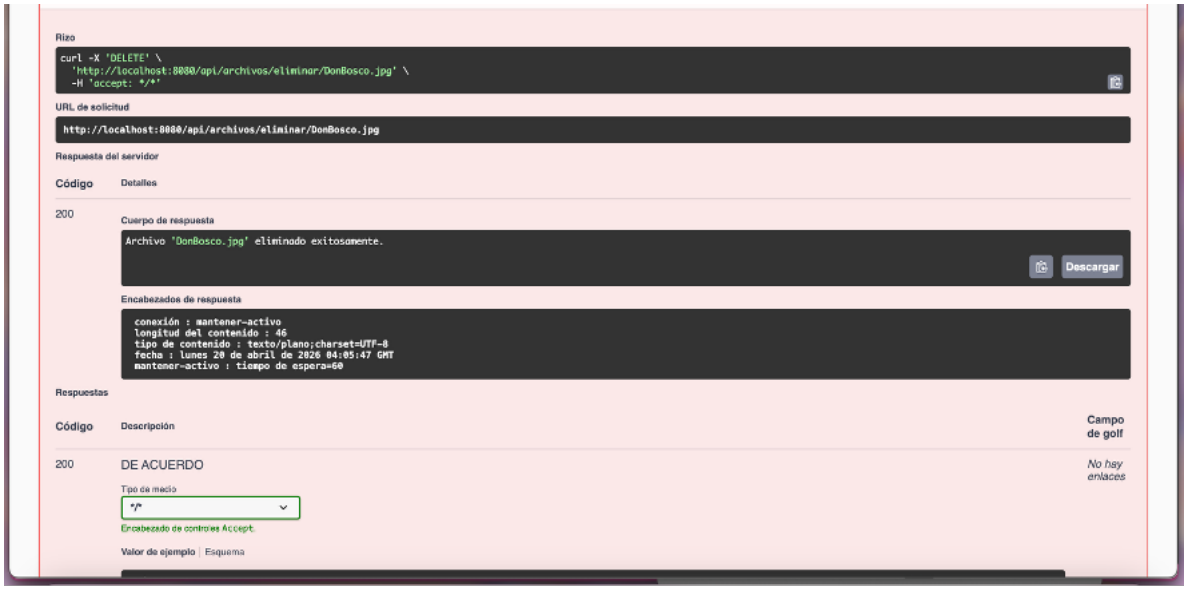
Se verificó la recuperación de bytes desde S3, asegurando que el archivo pueda ser descargado localmente en la máquina del usuario con su nombre original



Prueba de Eliminación (Delete)

Se ejecutó la eliminación del objeto, validando que tras la operación, el archivo ya no sea accesible ni para descarga ni para listado, optimizando el uso del espacio en el bucket.

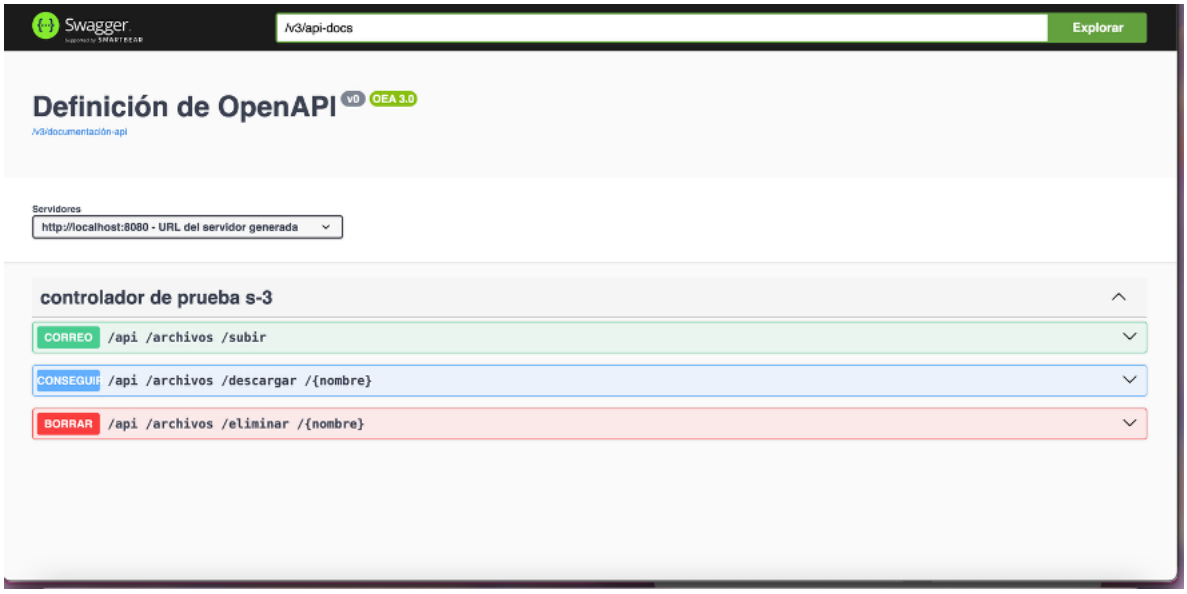




Resultados Obtenidos

Estabilidad: El sistema demostró ser capaz de manejar archivos binarios de forma consistente entre la aplicación Spring Boot y el contenedor de LocalStack.

Validación: Todas las respuestas HTTP devuelven códigos de estado adecuados (200 OK para éxitos y 404 Not Found cuando el archivo no existe), cumpliendo con los estándares de una API RESTful.


Swagger
/v3/api-docs
Explorar
Definición de OpenAPI
v0
OEA 3.0
/v3/documentación-api
Servidores
http://localhost:8080 - URL del servidor generada
controlador de prueba s-3
CORREO
/api /archivos /subir
CONSEGUIR
/api /archivos /descargar /{nombre}
BORRAR
/api /archivos /eliminar /{nombre}